Monitoring Log Schema Reference & Retention Rules

Document Reference: CAW-TECH-LOG-2026-01
System: Case Ace v2.0 Client Consultation Recording & Note Drafting System
Organisation: Citizens Advice Wandsworth (CAW)
Schema Implementation: backend/src/logging/logSchema.ts
Store Implementation: backend/src/logging/logStore.ts
Status: Formally Approved Technical Reference
Classification: Internal / Technical Reference

1. Privacy-Preserving Logging Philosophy

Under UK GDPR Article 5(1)© (Data Minimisation) and ICO guidance on AI telemetry, operational monitoring logs must be superficial only. They must provide high operational visibility for performance, security, and quality evaluation, while being completely useless to anyone attempting to extract client advice details or personal narratives.

Core Logging Invariants:

Strict Whitelist-Only Validation: Payloads are validated against a rigid TypeScript/JSON schema. Any unrecognised property causes an immediate rejection (LogSchemaValidationError).

Zero Free-Text Narrative: Free-text notes, client descriptions, transcripts, prompts, and Casebook client IDs are strictly prohibited.

Automated 365-Day Retention: Logs are retained for exactly 365 days (12 months) and purged automatically via a daily cron task.

Self-Auditing Access: Every query executed against the log store is itself recorded as an immutable LOGS_ACCESSED event.

2. Comprehensive Permitted Fields Specification

WHITELISTED LOG SCHEMA FIELDS		
Field Name	Data Type	Permitted Values / Constraints
`eventType`	String (Enum)	`CONSENT_GIVEN`, `CONSENT_WITHDRAWN`, `SESSION_STARTED`, `LOCAL_ASR_COMPLETED`, `REVIEW_GATE_ENTERED`, `REDACTIONS_CONFIRMED`, `ACOUSTIC_VERIFICATION_PASSED`, `CLOUD_DRAFT_COMPLETED`, `NOTE_SIGNED_OFF`, `SESSION_DESTROYED`, `MATERIAL_ERROR_FLAGGED`, `LOGS_ACCESSED`
`timestamp`	String (ISO 8601 UTC)	Regex: `^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}`
`pseudonymousUserId`	String (Regex)	Regex: `^usr_[a-zA-Z0-9_-]{4,32}$`
`role`	String (Enum)	`adviser`, `supervisor`, `auditor`, `admin`
`pseudonymousSessionId`	String (Regex)	Regex: `^ses_[a-zA-Z0-9_-]{4,32}$`
`intakeRoute`	String (Enum)	`live_in_person`, `webex_telephony`, `import`
`stageReached`	String (Enum)	`intake`, `recording`, `local_transcription`, `review_gate`, `cloud_drafting`, `sign_off`, `completed`
`stageDurationMs`	Integer	≥ 0 (Milliseconds)
`totalSessionDurationMs`	Integer	≥ 0 (Milliseconds)
`audioDurationSeconds`	Integer	≥ 0 (Seconds)
`wordCounts`	Integer	≥ 0 (Total word count of draft)
`detectedEntityCounts`	Object (Integer Map)	Keys: `names`, `ninos`, `postcodes`, `phones`, `specialCategory`; Values: ≥ 0
`adviserEntityOverrides`	Integer	Number of entities added/removed at gate (≥ 0)
`timeAtReviewGateSec`	Integer	≥ 0 (Seconds spent reviewing redactions)
`verificationPassResult`	String (Enum)	`passed`, `failed`, `aborted`
`modelAndPromptVersion`	String	e.g. `gemini-1.5-pro-002:v2.4.0`
`apiLatencyMs`	Integer	≥ 0 (Milliseconds)
`apiStatusCode`	Integer	HTTP Status (e.g. `200`, `429`, `500`)
`errorCode`	String (Enum / Code)	e.g. `ERR_ACOUSTIC_SURVIVOR`, `ERR_TIMEOUT`
`gapsAcknowledgedCount`	Integer	≥ 0 (Number of gaps checked by adviser)
`draftToSignOffSec`	Integer	≥ 0 (Seconds spent reviewing draft note)
`webexCallRef`	String (Regex)	Regex: `^call_[a-zA-Z0-9_-]{4,32}$`
`sourceEquipmentCategory`	String (Enum)	`dictaphone_olympus`, `dictaphone_sony`,

```
|                          |                       | `smartphone_memo`, `approved_outreach_mic`      |
+--------------------------------------------------------------------------------------------------------+
```

3. Explicitly Forbidden & Rejected Fields

The schema validation engine strictly rejects payloads containing any of the following fields or patterns:

Client Identifying Data: clientName, clientDob, nino, clientAddress, clientPostcode, clientPhone, clientEmail.

Casebook Data: casebookClientId, casebookCaseId, caseActivityId.

Consultation Narratives: transcript, rawTranscript, draftNote, noteContent, audioBlob, promptText.

Third-Party PII: landlordName, employerName, creditorName, familyMemberName.

4. Automated 365-Day Retention & Purge Implementation

The backend database store enforces automated time-to-live (TTL) pruning:

```
// backend/src/logging/logStore.ts
export class AuditLogStore {
  public purgeExpired(cutoffDateISO?: string): number {
    const cutoff = cutoffDateISO || new Date(Date.now() - 365 * 24 * 60 * 60 * 1000).toISOString();
    const initialCount = this.logs.length;
    this.logs = this.logs.filter((log) => log.timestamp >= cutoff);
    return initialCount - this.logs.length;
  }
}
```

Purge Execution: Executed automatically upon application boot and on a 24-hour recurring interval.

Non-Recoverable: Pruned records are permanently deleted from database memory and storage.